

Moduł sumatywny IOAD 2025/2026

Raport końcowy

Platforma PlayLink

Looking-For-Group dla gier niszowych, retro i mniej popularnych

Bartosz Kołaciński, nr indeksu: 251554

Mateusz Kosowski, nr indeksu: 251558

Nikodem Nowak, nr indeksu: 251598

Jakub Rosiak, nr indeksu: 251620

Jakub Rusek, nr indeksu: 247774

Opiekun projektu: mgr inż. Bartłomiej Jencz

19 czerwca 2026

Repozytorium: <https://github.com/ioad-modul-sumatywny-26/playlink>

Demo: <https://playlink.bartek.monster> • API:

<https://playlink-backend.bartek.monster/docs>

Spis treści

1	Lista wykonanych punktów	3
2	Analiza problemu	4
2.1	Pomysł na aplikację	4
2.2	Analiza istniejących rozwiązań	4
2.3	Wymagania funkcjonalne	5
2.4	Wymagania niefunkcjonalne i ich miary	6
2.5	Przypadki użycia i scenariusze	6
2.6	Diagram przypadków użycia	9
2.7	Wstępna architektura rozwiązania	9
2.8	Diagram komponentów	11
2.9	Wstępny model rozwiązania: warstwa danych i warstwa logiki	12
3	Projekt i implementacja rozwiązania	13
3.1	Stopień realizacji wymagań	13
3.2	Projekt graficznego interfejsu użytkownika	13
3.3	Aspekty AI/ML	14
3.4	Aspekty bezpieczeństwa danych	15
4	Testowanie rozwiązania	16
4.1	Charakterystyka rodzajów testów	17
4.2	Katalog wybranych przypadków testowych	17
4.3	Dokumentacja procesu i automatyzacja (CI)	21
5	Zarządzanie projektem	21
5.1	Cele i zakres projektu	21
5.2	Podział ról i zadań w zespole	22
5.3	Harmonogram realizacji i kamienie milowe	22
5.4	Szczegółowy harmonogram: kluczowe wskaźniki efektywności (KPI)	23
5.5	Narzędzia i proces pracy zespołowej	24

1 Lista wykonanych punktów

Poniższa tabela zestawia wszystkie kryteria oceny modułu sumatywnego ze stanem ich realizacji w projekcie PlayLink. Symbol ✓ oznacza punkt zrealizowany, a ✗ — punkt świadomie pominięty (zob. przypis).

Kryterium	Wyk.
<i>Analiza problemu</i>	
Analiza istniejących rozwiązań (min. 3) wraz z oceną zalet i wad (min. po 2)	✓
Określenie wymagań funkcjonalnych (min. 10)	✓
Określenie wymagań нефункциональных (min. 5) i ich miar	✓
Zidentyfikowanie przypadków użycia wraz ze scenariuszami	✓
Diagramy przypadków użycia	✓
Wstępna architektura rozwiązania	✓
Diagram komponentów	✓
Wstępny model rozwiązania (warstwa danych i warstwa logiki)	✓
<i>Projekt i implementacja</i>	
Prototyp w wersji podstawowej (min. 50% wymagań funkc. i нефункцик.)	✓
Prototyp w wersji zaawansowanej (min. 70% wymagań funkc. i нефункцик.)	✓
Prototyp w wersji zaawansowanej (min. 90% wymagań funkc. i нефункцик.)	✓
Projekt graficznego interfejsu użytkownika	✓
Uwzględnienie wybranych aspektów AI/ML [†]	✗
Uwzględnienie co najmniej 90% aspektów AI/ML [†]	✗
Uwzględnienie wybranych aspektów bezpieczeństwa danych	✓
Uwzględnienie co najmniej 90% aspektów bezpieczeństwa danych	✓
<i>Testowanie</i>	
Przeprowadzenie 3 różnych rodzajów testów + dokumentacja	✓
Przeprowadzenie 4 różnych rodzajów testów + dokumentacja	✓
Przeprowadzenie 5 różnych rodzajów testów + dokumentacja	✓
<i>Zarządzanie projektem</i>	
Przygotowanie opisu celów i zakresu projektu	✓
Określenie podziału ról i zadań w zespole	✓
Sporządzenie raportu końcowego ze wszystkimi wymaganymi elementami	✓
Harmonogram realizacji projektu z kamieniami milowymi	✓
Szczegółowy harmonogram z kamieniami milowymi i KPI	✓

[†] Aspekty AI/ML zostały — w uzgodnieniu z opiekunem projektu (mgr inż. Bartłomiej Jencz) — świadomie zastąpione rozszerzonym naciskiem na bezpieczeństwo danych (tożsamość bezhasłowa BIP39, podpisy ECDSA/EIP-191, ochrona przed atakami powtórzeniowymi). Uzasadnienie w rozdziale 3.4.

2 Analiza problemu

2.1 Pomysł na aplikację

PlayLink to lekka platforma webowa typu *LFG* (*Looking-For-Group*) służąca do natychmiastowego znajdowania współgraczy do gier niszowych, modów oraz tytułów retro. Projekt odpowiada na trzy konkretne problemy:

- **Martwe społeczności** — niszowe i stare gry mają zbyt mało aktywnych graczy, by samodzielnie utrzymać żywe lobby.
- **Brak dedykowanych narzędzi** — nie istnieje proste narzędzie do szybkiego zestawiania partnerów do takich gier.
- **Wysokie bariery wejścia** — istniejące platformy wymagają rejestracji, są skomplikowane lub nieaktywne.

Odpowiedzią PlayLink jest natychmiastowy dostęp *bez klasycznej rejestracji* (tożsamość BIP39), przeglądarka aktywnych sesji w czasie rzeczywistym oraz prosty mechanizm udostępniania pokoju linkiem.

2.2 Analiza istniejących rozwiązań

Przeanalizowano trzy najpopularniejsze alternatywy. Dla każdej wskazano co najmniej dwie zalety i dwie wady wraz z uzasadnieniem, dlaczego dana cecha jest istotna w kontekście gier niszowych. Zrzuty ekranu omawianych aplikacji znajdują się w prezentacji projektowej (załącznik).

Discord — kanały LFG na serwerach. Tekstowe kanały „looking-for-group” na serwerach społeczności konkretnych gier, z komunikacją głosową i tekstową w jednym miejscu.

- **Zalety:**
 - Bardzo szybka i bezpośrednia komunikacja (głos + tekst) — kluczowa przy umawianiu rozgrywki na żywo.
 - Ogromna, aktywna baza użytkowników — łatwo znaleźć ludzi do gier popularnych.
- **Wady:**
 - Brak dedykowanego systemu sesji — ogłoszenia toną w strumieniu wiadomości i szybko znikają, co uniemożliwia ich przeglądanie.
 - Trzeba znać i dołączyć do konkretnego serwera; brak wyszukiwania ofert *między* serwerami — przy grach niszowych właściwe kanały są martwe.

Reddit — r/LFG i subreddity gier. Tablice ogłoszeń „szukam grupy” w formacie postów na tematycznych subredditach.

- **Zalety:**
 - Duża baza użytkowników i prosty, znany format ogłoszeń.
 - Asynchroniczność — ogłoszenie można zostawić i wrócić do odpowiedzi później.
- **Wady:**
 - Całkowity brak komunikacji w czasie rzeczywistym — czeka się na komentarz, a posty starzeją się w godzinach, nie w minutach.

- Brak filtrowania po grze, regionie czy poziomie oraz brak struktury sesji; subreddity gier niszowych są często martwe.

FACEIT — matchmaking e-sportowy. Profesjonalna platforma e-sportowa z matchmakingiem, ligami i anti-cheatem dla gier kompetytywnych (CS2, Dota 2, League of Legends).

- **Zalety:**
 - Zaawansowany dobór graczy na podstawie umiejętności (skill-based matchmaking).
 - Duża baza graczy skupionych wokół rywalizacji oraz dedykowane serwery.
- **Wady:**
 - Obsługuje wyłącznie popularne tytuły e-sportowe — całkowite pominięcie gier niszowych i retro.
 - Wymaga instalacji klienta i anti-cheata, a część funkcji jest płatna — zbyt wysoka bariera dla casualowego gracza.

Podsumowanie. Żadne z istniejących rozwiązań nie łączy jednocześnie: (1) braku barier wejścia, (2) struktury sesji z aktualizacją w czasie rzeczywistym oraz (3) wsparcia dla gier niszowych. PlayLink przejmując najlepsze elementy konkurencji (czat sesyjny jak na Discordzie, otwartą listę ogłoszeń jak na Reddicie), eliminuje ich wady (trwała, przeszukiwalna lista pokoi z filtrowaniem; aktualizacja przez WebSocket) i dodaje nowe cechy: logowanie bezhasłowe BIP39 oraz kryptograficznie podpisywane wiadomości.

2.3 Wymagania funkcjonalne

Zidentyfikowano 15 wymagań funkcjonalnych; wszystkie zostały zrealizowane w prototypie.

ID	Wymaganie	Status
WF-01	Rejestracja i logowanie bezhasłowe (faza BIP39 + podpis ECDSA)	✓
WF-02	Odzyskiwanie konta przez import frazy mnemonicznej	✓
WF-03	Edycja nazwy użytkownika (z filtrem wulgaryzmów)	✓
WF-04	Tworzenie i konfiguracja pokoju (gra, limit graczy, lokalizacja)	✓
WF-05	Metadane pokoju (opis, link do komunikatora, wymagania)	✓
WF-06	Przeglądanie listy aktywnych pokoi na żywo	✓
WF-07	Dołączanie do pokoju i jego opuszczanie	✓
WF-08	Czat tekstowy w obrębie pokoju (w czasie rzeczywistym)	✓
WF-09	Kryptograficzne podpisywanie wiadomości czatu (EIP-191)	✓
WF-10	Wyszukiwanie pokoi i użytkowników	✓
WF-11	Filtrowanie pokoi po grze oraz sortowanie listy	✓
WF-12	Kalendarz pokoju i RSVP dla zaplanowanej sesji	✓
WF-13	Własne gry (custom games) i ping zależny od lokalizacji lobby	✓
WF-14	Panel administratora: zarządzanie pokojami i kategoriami gier	✓

ID	Wymaganie	Status
WF-15	Moderacja: wyrzucenie uczestnika z pokoju (kick) przez admina	✓

2.4 Wymagania niefunkcjonalne i ich miary

ID	Wymaganie	Miara / kryterium akceptacji	St.
WN-01	Wydajność	Czas odpowiedzi API < 200 ms dla 95% żądań (p95)	✓
WN-02	Responsywność realtime	Aktualizacja UI po zdarzeniu < 1000 ms	✓
WN-03	Dostępność	Uptime $\geq$ 99% w skali miesiąca	✓
WN-04	Skalowalność	Do 128 użytkowników jednocześnie w pokoju	✓
WN-05	Niezawodność	Docker healthcheck + automatyczny restart kontenera	✓
WN-06	Realtime / odporność	WebSocket z heartbeatem 30 s i auto-reconnectem	✓
WN-07	Bezpieczeństwo transportu	TLS 1.3 dla HTTPS i WSS (Let's Encrypt)	✓
WN-08	Ochrona przed nadużyciami	Rate limiting: 10/min (REST), 10 wiad./10 s (czat WS)	✓
WN-09	Użyteczność	Utworzenie pokoju w $\leq$ 30 s bez instrukcji	✓
WN-10	Modułowość	Frontend i backend wdrażane niezależnie	✓
WN-11	Jakość kodu	Pokrycie testami backendu $\geq$ 80% (bramka CI)	✓
WN-12	Wdrażalność	Czas od push do wersji live $\approx$ 3 min	✓

2.5 Przypadki użycia i scenariusze

W systemie występują czterej aktorzy: **Gość** (niezalogowany), **Użytkownik** (zalogowany, dziedziczy uprawnienia Gościa), **Administrator** (dziedziczy uprawnienia Użytkownika) oraz **System** (zadania automatyczne — weryfikacja challenge-response i czyszczenie wygasłych pokoi).

Lista przypadków użycia.

- **Gość:** przeglądanie listy pokoi; wyszukiwanie i filtrowanie; rejestracja / logowanie.
- **Użytkownik:** tworzenie i konfiguracja pokoju; dołączanie/opuszczanie; czat (podpisany); planowanie wydarzenia i RSVP; edycja nazwy; odzyskiwanie konta.
- **Administrator:** wyrzucenie uczestnika (kick); zarządzanie kategoriami gier.

- **System:** weryfikacja podpisu i wydanie JWT; czyszczenie wygasłych pokoi i wiadomości.

Diagram przypadków użycia przedstawiono na rys. 1. Poniżej opisano scenariusze pięciu kluczowych przypadków użycia — po jednym opracowanym przez każdego członka zespołu — w ujęciu: przepływ główny oraz przepływ alternatywny.

UC-1: Rejestracja i pierwsze logowanie (BIP39). *(autor: Nikodem Nowak)*

- *Aktor:* Gość. *Warunek wstępny:* brak.
- *Przepływ główny:*
 1. Gość wybiera „Enter realm” i generuje nową 12-słowną frazę BIP39 (klucz prywatny derywowany lokalnie, nigdy nie opuszcza przeglądarki).
 2. Klient pobiera z backendu jednorazowy *nonce* (POST /auth/request-nonce).
 3. Klient podpisuje komunikat zawierający nonce (*personal_sign*, EIP-191).
 4. Backend odtwarza adres z podpisu, weryfikuje nonce i wydaje token JWT (POST /auth/verify).
 5. BFF zapisuje JWT w ciasteczku *httpOnly*; użytkownik jest zalogowany.
- *Przepływ alternatywny:* nonce wygasł lub został już użyty — backend odrzuca weryfikację (HTTP 400/401), klient ponawia od kroku 2.

UC-2: Utworzenie pokoju i rozpoczęcie czatu. *(autor: Bartosz Kołaciński)*

- *Aktor:* Użytkownik. *Warunek wstępny:* zalogowany; mniej niż 3 aktywne pokoje.
- *Przepływ główny:*
 1. Użytkownik wypełnia formularz (nazwa, gra, limit graczy, lokalizacja, opcjonalnie opis/link/wymagania) i zatwierdza.
 2. Backend tworzy pokój, dołącza twórcę jako członka i rozgłasza nową listę pokoi przez kanał /ws/rooms.
 3. Użytkownik wchodzi do pokoju; otwiera się kanał czatu /ws/rooms/<name>/chat.
- *Przepływ alternatywny:* przekroczony limit 3 pokoi lub zajęta nazwa — backend zwraca błąd walidacji, formularz wyświetla komunikat.

UC-3: Dołączenie do pokoju i deklaracja RSVP. *(autor: Jakub Rosiak)*

- *Aktor:* Użytkownik. *Warunek wstępny:* pokój istnieje i nie jest pełny.
- *Przepływ główny:*
 1. Użytkownik wybiera pokój z listy i dołącza; roster jest rozgłaszany do pozostałych członków.
 2. Jeśli pokój ma zaplanowane wydarzenie, użytkownik deklaruje obecność (*present / maybe / absent*); zdarzenie RSVP jest rozgłaszane.
 3. Użytkownik bierze udział w czacie pokoju.
- *Przepływ alternatywny:* pokój pełny lub wygasł — dołączenie zostaje odrzucone, użytkownik wraca do listy.

UC-4: Moderacja — wyrzucenie uczestnika. *(autor: Jakub Rusek)*

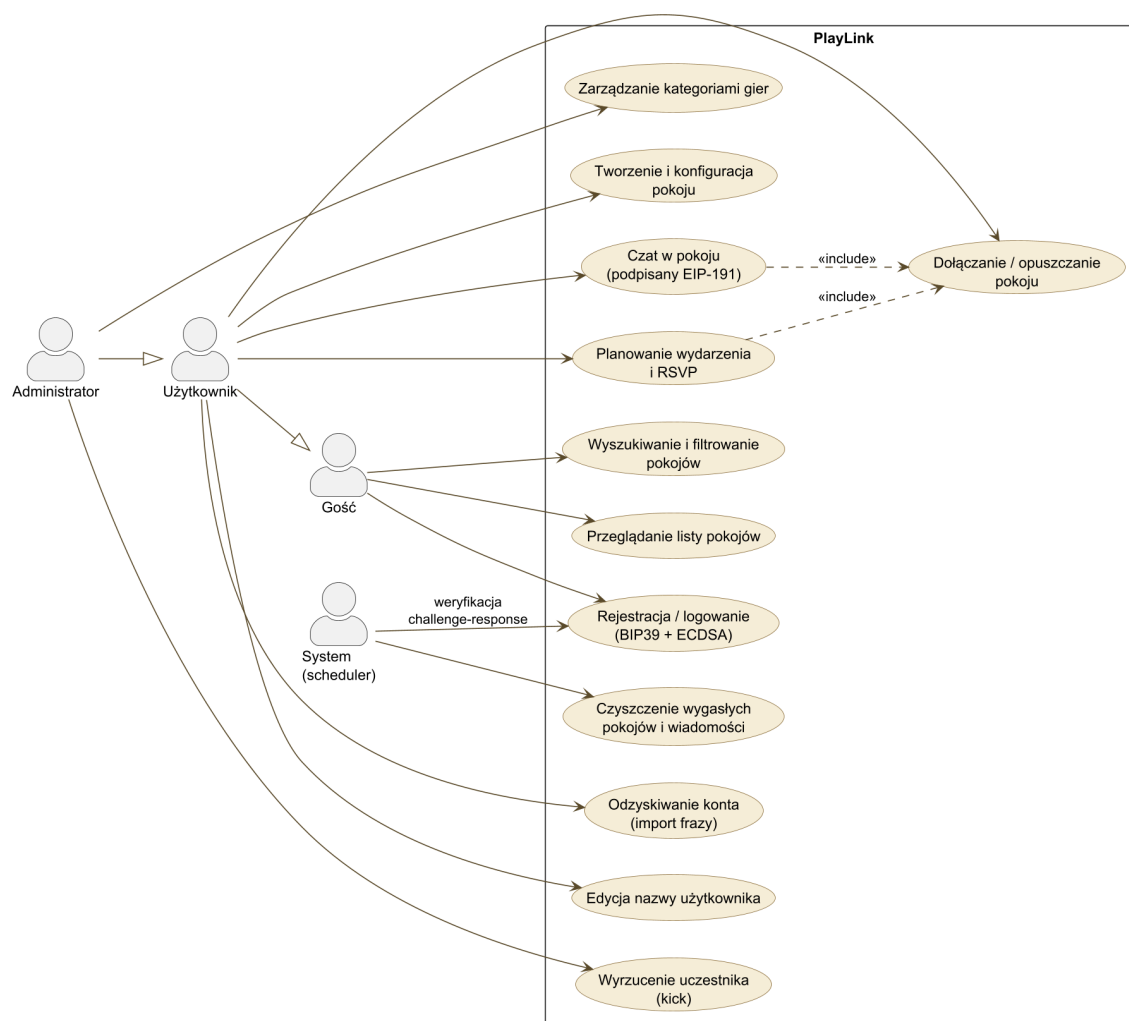
- *Aktor:* Administrator. *Warunek wstępny:* adres administratora znajduje się w ADMIN_ADDRESSES.
- *Przepływ główny:*
 1. Administrator wybiera uczestnika i akcję „kick”.

2. Backend weryfikuje uprawnienia, usuwa członkostwo i rozgłasza ramkę *kick* oraz zaktualizowany roster; wyrzucony klient zostaje rozłączony.
- *Przeptyw alternatywny*: żądanie pochodzi od konta bez uprawnień administratora — backend odrzuca operację (HTTP 403).

UC-5: Wyszukiwanie i filtrowanie pokoiów. (*autor: Mateusz Kosowski*)

- *Aktor*: Gość lub Użytkownik. *Warunek wstępny*: brak.
- *Przeptyw główny*:
 1. Aktor otwiera listę pokoiów, która jest na bieżąco aktualizowana przez kanał `/ws/rooms`.
 2. Aktor wpisuje frazę wyszukiwania i/lub wybiera filtr gry oraz porządek sortowania.
 3. Lista zawęży się do pokoiów spełniających kryteria; wybór pokoju prowadzi do jego szczegółów (a po zalogowaniu — do dołączenia, UC-2/UC-3).
- *Przeptyw alternatywny*: brak pokoiów spełniających kryteria — wyświetlany jest stan pusty z podpowiedzią rozszerzenia filtrów lub utworzenia własnego pokoju.

2.6 Diagram przypadków użycia



Rysunek 1: Diagram przypadków użycia PlayLink (aktorzy: Gość, Użytkownik, Administrator, System).

2.7 Wstępna architektura rozwiązania

PlayLink ma architekturę trójwarstwową z wzorcem **BFF** (*Backend-for-Frontend*). Przeglądarka komunikuje się *bezpośrednio* z API (REST oraz dwa kanały WebSocket) dla większości odczytów i całego ruchu realtime, a z serwerem SvelteKit (BFF) tylko dla operacji wymagających dostępu do ciasteczka sesyjnego `httpOnly` (logowanie/wylogowanie oraz uwierzytelnione akcje formularzy), które BFF proxy-uje do backendu z dołączonym tokenem JWT.

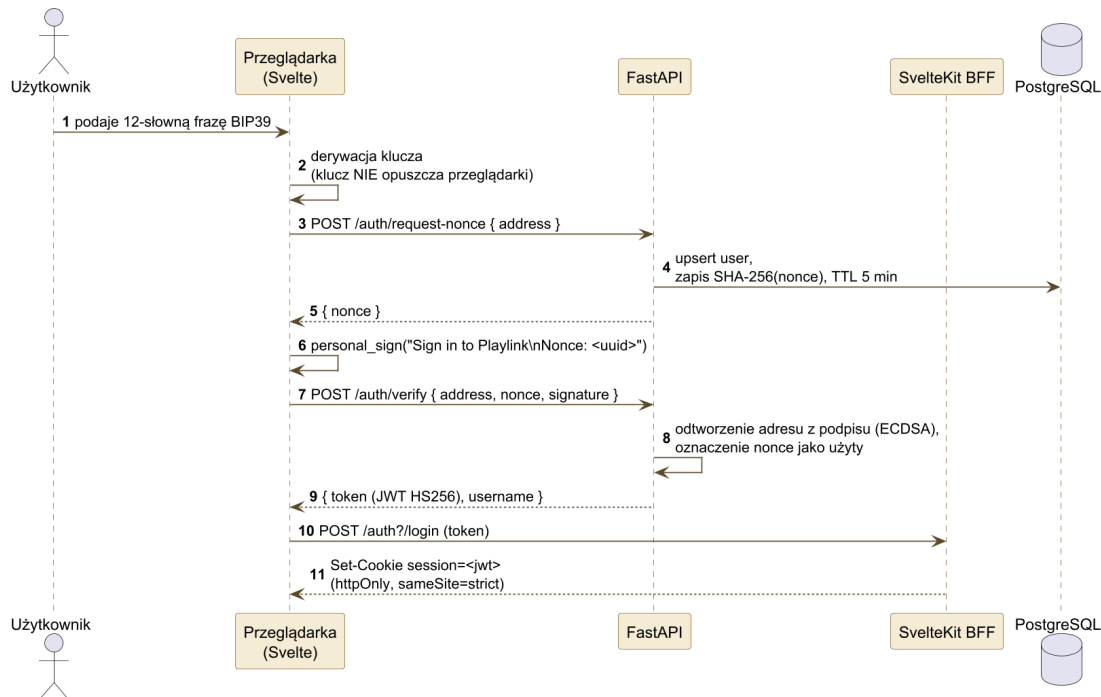
- **Wzorce projektowe:** BFF (izolacja JWT od JavaScriptu klienta), wstrzykiwanie zależności (FastAPI Depends), wzorec repozytorium przez SQLAlchemy, publish-subscribe dla rozgłaszania zdarzeń WebSocket.
- **Główne moduły:** klient Svelte (podpisywanie BIP39 przez ethers/viem), BFF SvelteKit (adapter-node), API FastAPI (REST + WebSocket + zadanie w tle), baza PostgreSQL.

- Stos technologiczny zestawiono w tab. 4.

Warstwa	Technologia
Frontend	SvelteKit 2 / Svelte 5 (runes), TypeScript, Tailwind CSS (build: Bun)
Podpisywanie	ethers + viem (BIP39 + EIP-191)
Backend	FastAPI (Python 3.14+), serwer ASGI (build: uv)
ORM / migracje	SQLModel (SQLAlchemy + Pydantic), Alembic
Baza danych	PostgreSQL 18 (prod) / SQLite in-memory (testy)
Realtime	FastAPI WebSockets (kanały: rooms, chat)
Uwierzytelnianie	tożsamość BIP39 + ECDSA (<code>eth_account</code>), JWT HS256
Rate limiting	sloapi (REST) + przesuwne okno per-adres (czat)
Orkiestracja	Docker Compose (db, backend, frontend), Nginx, Tailscale
CI/CD	GitHub Actions → deploy SSH przez Tailscale

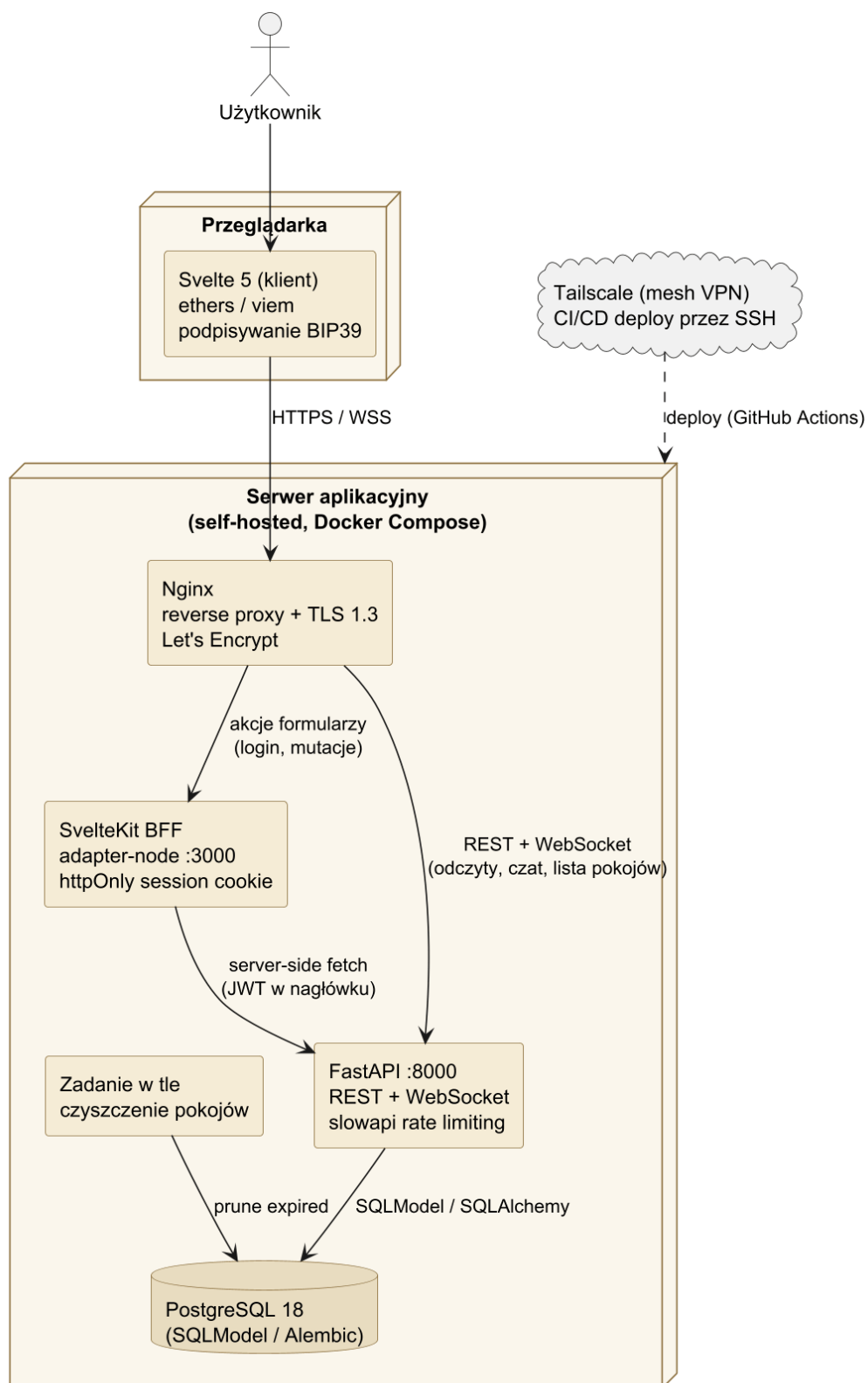
Tabela 4: Stos technologiczny PlayLink.

Przepływ uwierzytelniania challenge-response przedstawia rys. 2.



Rysunek 2: Diagram sekwencji: bezhasłowe logowanie (challenge-response BIP39/ECDSA). Klucz prywatny nigdy nie opuszcza przeglądarki; JWT trafia do ciasteczka `httpOnly` ustawianego przez BFF.

2.8 Diagram komponentów



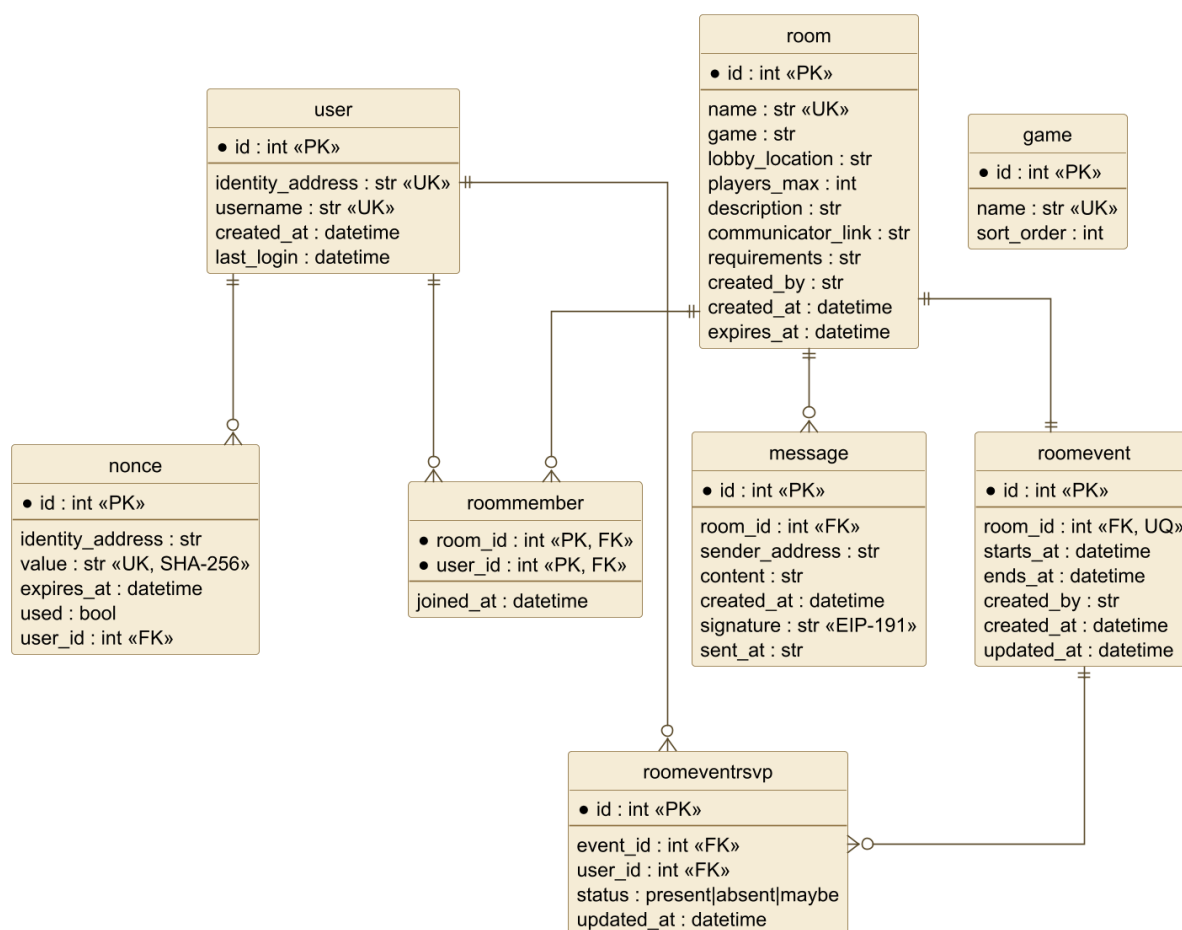
Rysunek 3: Diagram komponentów: przeglądarka, Nginx (TLS), BFF SvelteKit, API FastAPI z zadaniem czyszczącym oraz baza PostgreSQL; wdrożenie przez Tailscale.

2.9 Wstępny model rozwiązania: warstwa danych i warstwa logiki

Warstwa danych

Trwałość zapewnia PostgreSQL z ośmioma encjami zarządzanymi przez SQLAlchemy i migracjami Alembic. Model danych przedstawia diagram ERD na rys. 4.

- **user** — tożsamość: **identity_address** (adres EIP-55 odtworzony z ECDSA) oraz unikalna **username**; *brak hasła*.
- **nonce** — jednorazowe, krótkożyciowe wyzwania; w bazie przechowywany jest wyłącznie skrót SHA-256 wartości nonce (**value**), z flagą **used**.
- **room** — sesja gry: nazwa (unikalna), gra, lokalizacja lobby, limit graczy, metadane oraz **expires_at** (TTL, domyślnie 60 min).
- **roommember** — tabela łącząca (M:N) użytkowników i pokoje (klucz złożony).
- **message** — wiadomości czatu; opcjonalny podpis EIP-191 (**signature**) i podpisany znacznik czasu (**sent_at**) zapewniają weryfikowalne autorstwo.
- **roomevent** / **roomeventsvp** — zaplanowane wydarzenie pokoju oraz deklaracje obecności (status *present/absent/maybe*, semantyka upsert).
- **game** — katalog kategorii gier (kolejność wg **sort_order**).



Rysunek 4: Model danych (ERD) PlayLink — osiem encji wraz z kluczami i relacjami.

Warstwa logiki

Logika backendu jest skupiona w usłudze FastAPI udostępniającej REST oraz dwa kanały WebSocket. Najważniejsze elementy:

- **REST** — endpointy uwierzytelniania (`/auth/request-nonce`, `/auth/verify`), CRUD pokoi, profil użytkownika, administracja. Dokumentacja generowana automatycznie (Swagger/OpenAPI).
- **WebSocket** — kanał `/ws/rooms` (rozgłasza pełną listę pokoi po każdej mutacji) oraz `/ws/rooms/<name>/chat` (czat z autoryzacją JWT i kontrolą członkostwa; ramki: event, RSVP, roster, room-closed, kick).
- **Logika biznesowa** — m.in.: limit 3 aktywnych pokoi na użytkownika, weryfikacja podpisów EIP-191 z odrzuceniem nadmiernego dryfu czasu (`CHAT_SIGNATURE_SKEW_SECONDS`), upsert RSVP, dobór pinga na podstawie lokalizacji lobby (odległość Haversine).
- **Zadanie w tle** — cykliczne czyszczenie wygasłych pokoi i wiadomości (co `ROOM_CLEANUP_INTERVAL`).

3 Projekt i implementacja rozwiązania

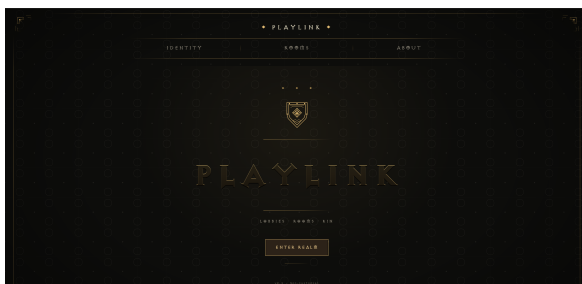
3.1 Stopień realizacji wymagań

Prototyp PlayLink osiągnął poziom **wersji zaawansowanej** ($\geq 90\%$). Wszystkie 15 wymagań funkcjonalnych z tab. 2.3 oraz wszystkie 12 wymagań нефункциональных z tab. 2.4 zostały zrealizowane i wdrożone na środowisku produkcyjnym (<https://playlink.bartek.monster>). Aplikacja jest w pełni działająca: uwierzytelnianie BIP39, tworzenie/dołączanie/opuszczanie pokoi, czat realtime z podpisami, kalendarz i RSVP, wyszukiwanie i filtrowanie, panel administratora oraz automatyczne czyszczenie wygasłych pokoi.

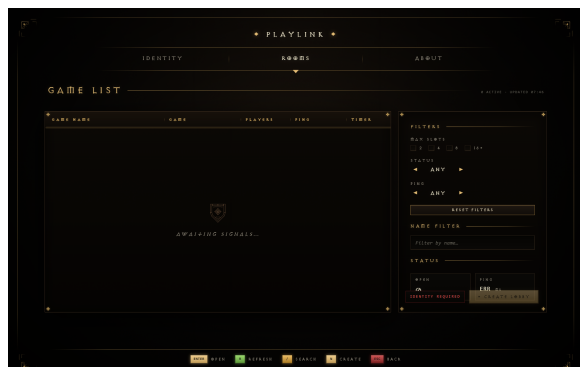
Zastosowano aktualne standardy projektowe: typowanie statyczne w całym stosie (TypeScript + Pydantic/SQLModel), automatyczną walidację żądań, migracje bazy (Alembic) oraz w pełni zautomatyzowany pipeline CI/CD (rozdz. 5).

3.2 Projekt graficznego interfejsu użytkownika

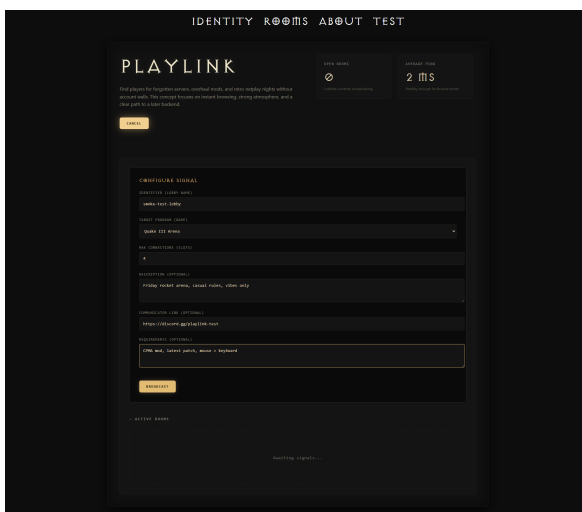
Interfejs zaprojektowano w estetyce inspirowanej przeglądarką lobby z gry *Diablo II: Resurrected* — ciemna, gotycka paleta złoto-brązowa i minimalistyczny układ. Frontend zbudowano w SvelteKit 2 / Svelte 5 (runes) z Tailwind CSS; UI reaguje natychmiast na zdarzenia WebSocket. Poniżej wybrane widoki aplikacji.



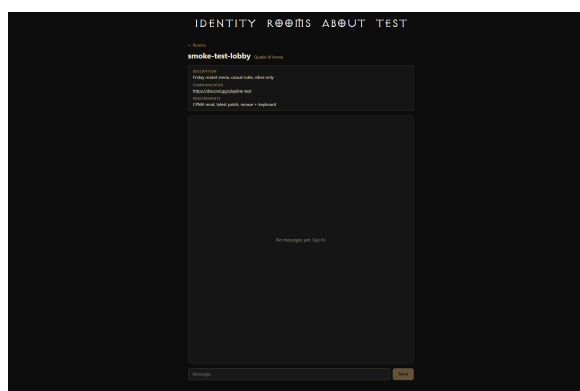
(a) Strona startowa („Enter realm”).



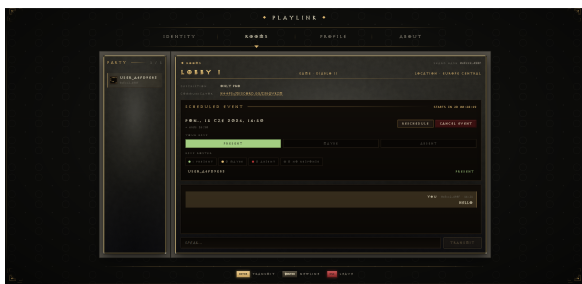
(b) Przeglądarka pokoiów (lista lobby).



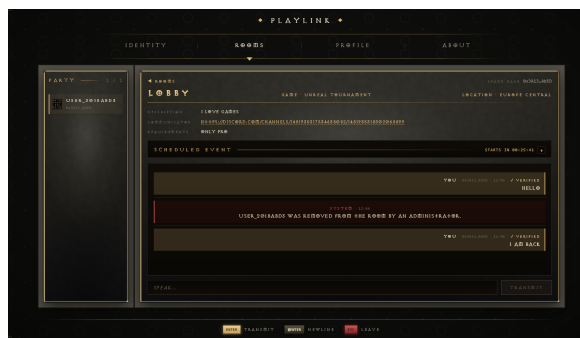
(c) Formularz tworzenia pokoju.



(d) Szczegóły pokoju z metadanymi.



(e) Widok pokoju: uczestnicy, wydarzenie z RSVP, czat.



(f) Moderacja: zdarzenie systemowe o wyrzuceniu uczestnika.

Rysunek 5: Wybrane widoki graficznego interfejsu PlayLink.

3.3 Aspekty AI/ML

W uzgodnieniu z opiekunem projektu (mgr inż. Bartłomiej Jencz) zespół świadomie zrezygnował z komponentu AI/ML na rzecz **rozszerzonego, dopracowanego modelu bezpieczeństwa danych** (rozdz. 3.4). Decyzja wynika z charakteru produktu: kluczową wartością PlayLink jest natychmiastowy, bezpieczny i prywatny dostęp (tożsamość niepowiernicza BIP39, weryfikowalne autorstwo wiadomości), a nie rekomendacje oparte na uczeniu maszynowym. Dobór pinga odbywa się deterministycznie na podstawie odległości

geograficznej lobby (metryka Haversine), bez modelu ML.

3.4 Aspekty bezpieczeństwa danych

Bezpieczeństwo jest centralnym filarem PlayLink. Przyjęto zasadę *non-custodial*: klucz prywatny nigdy nie opuszcza przeglądarki, a backend wyłącznie weryfikuje dowody kryptograficzne. Poniżej uzgodnione aspekty bezpieczeństwa — wszystkie zrealizowane (pokrycie $\geq 90\%$).

1. Tożsamość bezhasłowa (BIP39 + ECDSA). Konto reprezentuje 12-słowna fraza mnemoniczna BIP39 (standard z Bitcoin, 2013; 2048-wyrazowy słownik daje 132 bity entropii). Z frazy derywowany jest klucz secp256k1, a publiczny adres EIP-55 stanowi identyfikator konta. Eliminuje to e-mail, numer telefonu i hasło — a tym samym wektor ataku „reset hasła”.

2. Uwierzytelnianie challenge-response i ochrona przed replay. Logowanie wykorzystuje jednorazowy *nonce*: backend wydaje wyzwanie, klient podpisuje je (EIP-191 `personal_sign`), a backend odtwarza adres z podpisu (ECDSA recovery). Zabezpieczenia:

- *nonce* jest **jednorazowy** (flaga `used`) i **krótkożyty** (TTL 5 min);
- w bazie przechowywany jest wyłącznie **skrót SHA-256** wartości *nonce* — surowa wartość nigdy nie trafia na dysk;
- ponowne wydanie *nonce* unieważnia poprzedni — co blokuje ataki powtórzeniowe.

3. Izolacja tokenu sesji (wzorzec BFF, hardening XSS). Token JWT (HS256) jest przechowywany w ciasteczku `httpOnly, sameSite=strict`, ustawianym przez serwer SvelteKit. Token **nigdy nie jest dostępny dla JavaScriptu klienta**, co znacząco ogranicza skutki ewentualnego XSS.

4. Weryfikowalne autorstwo wiadomości. Wiadomości czatu są podpisywane kluczem derywowanym z BIP39 (EIP-191). Serwer rekonstruuje kanoniczną postać wiadomości i weryfikuje podpis, a znaczniki czasu dryfujące ponad `CHAT_SIGNATURE_SKEW_SECONDS` są odrzucane — co ogranicza powtórzenia. Autorstwo jest weryfikowalne *end-to-end*, a nie tylko na podstawie JWT.

5. Autoryzacja administratora bez eskalacji. Uprawnienia administratora wynikają wyłącznie z listy adresów w zmiennej środowiskowej `ADMIN_ADDRESSES` — w bazie nie istnieje kolumna `is_admin`. Uniemożliwia to eskalację uprawnień przez modyfikację danych; sfałszowane roszczenie o admina jest odrzucane (zob. testy bezpieczeństwa, rozdz. 4).

6. Ograniczanie nadużyć (rate limiting) i higiena treści. REST jest limitowany przez słowapi (domyślnie 10/min), a czat — przesuwным oknem per-adres. Edycja nazwy użytkownika przechodzi przez filtr wulgaryzmów (stoplist).

7. Bezpieczeństwo sieci i wdrożenia. Cały ruch szyfrowany jest TLS 1.3 (HTTPS + WSS, certyfikaty Let’s Encrypt). Wdrożenie jest self-hosted: transfer artefaktów odbywa się prywatnym kanałem przez Tailscale (mesh VPN, WireGuard), publiczny SSH nie jest wystawiony (brak ataków brute-force na port 22), a baza PostgreSQL nie ma mapowania portu na hosta.

Weryfikacja. Wszystkie powyższe mechanizmy są pokryte automatycznymi testami bezpieczeństwa (replay nonce, wygasły nonce, sfalszowane roszczenie admina, zmanipulowane podpisy) — zob. rozdz. 4.

4 Testowanie rozwiązania

PlayLink jest pokryty obszernym, zautomatyzowanym zestawem testów: około **133 testy** po stronie backendu (pytest) oraz około **60 testów** po stronie frontendu (Vitest). Pod względem *różnorodności* wyróżniono **sześć różnych rodzajów testów** (powyżej wymaganego minimum pięciu), zestawionych w tab. 5. W podrozdziale 4.2 opisano ponadto **dziesięć reprezentatywnych przypadków testowych** (po kilka na rodzaj) wraz z celem, metodą i wynikiem.

Uwaga terminologiczna (różnorodność vs. liczba). Kryterium oceny dotyczy *rodzajów* (typów) testów — wymaganych jest 5 różnych. Liczba opisanych przypadków (10) to inna oś: są to konkretne testy rozłożone na owe rodzaje (np. 2 na rodzaj). Oba wymagania są zatem spełnione jednocześnie: 6 rodzajów oraz 10 szczegółowo udokumentowanych przypadków.

Rodzaj testów	Narzędzie	Zakres	Liczba
Jednostkowe	pytest, Vitest	Funkcje pomocnicze, walidacja, store’y, metryka Haversine	~25
Integracyjne / API	FastAPI TestClient	Endpointy REST przez całą aplikację (SQLite in-memory)	~54
WebSocket / real-time	TestClient WS	Rozgłaszanie ramek czatu i zdarzeń (34 połączenia WS)	~45
Bezpieczeństwa	pytest eth_account	Replay, wygasły nonce, fałszywy admin, podrobione podpisy	~10
Migracji (smoke)	pytest + Alembic	Migracja do head, weryfikacja schematu; w CI round-trip na Postgresie	1 (+CI)
Kontraktowe (frontend)	Vitest	Server-loadery i akcje SvelteKit (+page.server.ts)	~20

Tabela 5: Sześć rodzajów testów w projekcie PlayLink.

4.1 Charakterystyka rodzajów testów

Testy jednostkowe. Weryfikują izolowaną logikę bez bazy i sieci: parsowanie listy adresów administratorów, funkcję `hash_nonce`, walidację nazw użytkownika, a we frontendzie — podpisywanie, store’y oraz obliczanie odległości lobby (Haversine). Narzędzia: pytest, Vitest + jsdom.

Testy integracyjne / API. Uruchamiają żądania HTTP przez całą aplikację FastAPI z użyciem `TestClient` i bazy SQLite in-memory (fixture nadpisuje `get_session`). Pokrywają uwierzytelnianie, pełny cykl życia pokoju, panel administratora i akcję kick — sprawdzając kody statusu oraz stan w bazie.

Testy WebSocket / realtime. Wykorzystują `TestClient.websocket_connect` (34 połączenia WS) do weryfikacji rozgłaszania ramek: *event-update*, *rsvp-update*, *roster-update*, *room-closed*, *kick*.

Testy bezpieczeństwa. Najważniejszy rodzaj dla profilu projektu — scenariusze ataków (replay, wygasły nonce, eskalacja uprawnień, podrobione podpisy EIP-191), z realnym podpisywaniem kluczem (`eth_account`).

Testy migracji (smoke). Aplikują pełny łańcuch Alembic (`alembic upgrade head`) i sprawdzają obecność wszystkich ośmiu tabel; w CI powtarzane na realnym PostgreSQL z round-tripem `downgrade/upgrade` i `alembic check`.

Testy kontraktowe frontendu. Importują server-loadery i akcje SvelteKit z zamockowanymi `fetch/cookies`, weryfikując przekierowania, strażników autoryzacji i walidację formularzy.

4.2 Katalog wybranych przypadków testowych

Poniżej opisano dziesięć reprezentatywnych przypadków testowych w jednolitym formacie (rodzaj, cel, metoda/narzędzie, dane/kroki, oczekiwany wynik, rezultat). Wszystkie przechodzą pomyślnie w pipeline CI; zespół może dołączyć do każdego zrzut ekranu z wynikiem (np. raport pytest/Vitest lub log CI).

TC-01: Parsowanie adresów administratorów

<i>Rodzaj</i>	Jednostkowy
<i>Cel</i>	Sprawdzić poprawne wczytanie i normalizację listy adresów z ADMIN_ADDRESSES.
<i>Metoda / narzędzie</i>	pytest (test_helpers.py, _parse_admin_addresses).
<i>Dane / kroki</i>	Wejście " 0xABC , 0xdef " (spacje, różna wielkość liter).
<i>Oczekiwany wynik</i>	Zbiór znormalizowanych adresów małymi literami: {0xabc, 0xdef}.
<i>Rezultat</i>	PASS

TC-02: Hashowanie wartości nonce

<i>Rodzaj</i>	Jednostkowy
<i>Cel</i>	Zapewnić, że nonce jest deterministycznie hashowany (SHA-256) przed zapisem.
<i>Metoda / narzędzie</i>	pytest (hash_nonce).
<i>Dane / kroki</i>	Ten sam nonce hashowany dwukrotnie oraz dwa różne nonce.
<i>Oczekiwany wynik</i>	Identyczny skrót dla identycznego wejścia; różne skróty dla różnych wejść.
<i>Rezultat</i>	PASS

TC-03: Obliczanie odległości lobby (Haversine)

<i>Rodzaj</i>	Jednostkowy (frontend)
<i>Cel</i>	Zweryfikować dobór pingu na podstawie odległości geograficznej lobby.
<i>Metoda / narzędzie</i>	Vitest (lobbyLocations.test.ts).
<i>Dane / kroki</i>	Pary współrzędnych o znanej odległości.
<i>Oczekiwany wynik</i>	Odległość w granicach tolerancji; właściwy przedział pingu.
<i>Rezultat</i>	PASS

TC-04: Pełny przepływ logowania (challenge-response)

<i>Rodzaj</i>	Integracyjny / API
<i>Cel</i>	Zweryfikować ścieżkę request-nonce → verify → wydanie JWT.
<i>Metoda / narzędzie</i>	FastAPI TestClient (test_auth.py), podpis eth_account.
<i>Dane / kroki</i>	POST /auth/request-nonce, podpis nonce, POST /auth/verify.
<i>Oczekiwany wynik</i>	HTTP 200 oraz poprawny token JWT i nazwa użytkownika w odpowiedzi.
<i>Rezultat</i>	PASS

TC-05: Cykl życia pokoju i limit aktywnych pokoi

<i>Rodzaj</i>	Integracyjny / API
<i>Cel</i>	Sprawdzić tworzenie, dołączanie, opuszczanie oraz limit 3 aktywnych pokoi.
<i>Metoda / narzędzie</i>	FastAPI TestClient (test_rooms_lifecycle.py).
<i>Dane / kroki</i>	Sekwencja create/join/leave; próba utworzenia 4. pokoju.
<i>Oczekiwany wynik</i>	Operacje 1–3 kończą się sukcesem; 4. pokój odrzucony błędem walidacji.
<i>Rezultat</i>	PASS

TC-06: Rozgłaszanie wiadomości i ramek w czacie

<i>Rodzaj</i>	WebSocket / realtime
<i>Cel</i>	Potwierdzić, że klienci otrzymują aktualizacje w czasie rzeczywistym.
<i>Metoda / narzędzie</i>	TestClient.websocket_connect (test_chat.py, test_room_events.py).
<i>Dane / kroki</i>	Dwa połączenia WS w jednym pokoju; wysłanie wiadomości i zmiana RSVP.
<i>Oczekiwany wynik</i>	Obaj klienci otrzymują ramki message, rsvp-update, roster-update.
<i>Rezultat</i>	PASS

TC-07: Odrzucenie ataku powtórzeniowego (replay)

<i>Rodzaj</i>	Bezpieczeństwa
<i>Cel</i>	Uniemożliwić ponowne użycie tego samego nonce.
<i>Metoda / narzędzie</i>	pytest (<code>test_verify_signature_rejects_replayed_nonce</code>).
<i>Dane / kroki</i>	Poprawna weryfikacja, a następnie ponowna z tym samym nonce/podpisem.
<i>Oczekiwany wynik</i>	Pierwsza weryfikacja: 200; druga: odrzucenie (nonce oznaczony jako użyty).
<i>Rezultat</i>	PASS

TC-08: Brak eskalacji uprawnień administratora

<i>Rodzaj</i>	Bezpieczeństwa
<i>Cel</i>	Sfałszowane roszczenie o rolę admina nie może dawać dostępu.
<i>Metoda / narzędzie</i>	pytest (<code>test_forged_admin_claim_does_not_grant_admin_access</code>).
<i>Dane / kroki</i>	Token/żądanie konta spoza ADMIN_ADDRESSES próbuje akcji administracyjnej.
<i>Oczekiwany wynik</i>	HTTP 403; operacja administracyjna nie zostaje wykonana.
<i>Rezultat</i>	PASS

TC-09: Migracja schematu bazy (smoke)

<i>Rodzaj</i>	Migracji
<i>Cel</i>	Zapewnić, że pełny łańcuch migracji buduje poprawny schemat.
<i>Metoda / narzędzie</i>	pytest + Alembic (<code>test_migrations.py</code>); w CI na PostgreSQL.
<i>Dane / kroki</i>	<code>alembic upgrade head</code> na świeżej bazie; inspekcja schematu.
<i>Oczekiwany wynik</i>	Obecnych wszystkie 8 tabel i kluczowe kolumny; brak dryfu (<code>alembic check</code>).
<i>Rezultat</i>	PASS

TC-10: Strażnik autoryzacji trasy (frontend)

<i>Rodzaj</i>	Kontraktowy (frontend)
<i>Cel</i>	Niezałogowany użytkownik nie ma dostępu do tras chronionych.
<i>Metoda / narzędzie</i>	Vitest (<code>rooms.name.page.server.test.ts</code>), zamockowane <code>cookies/fetch</code> .
<i>Dane / kroki</i>	Wywołanie <code>load</code> bez ważnej sesji.
<i>Oczekiwany wynik</i>	Przekierowanie HTTP 303 do <code>/rooms</code> ; brak ujawnienia danych chronionych.
<i>Rezultat</i>	PASS

4.3 Dokumentacja procesu i automatyzacja (CI)

Proces testowy jest opisany w repozytorium (`docs/operations/testing.md` — tabele pokrycia per plik, opis fixture’ów i kanałów WS). Konfiguracja: `pyproject.toml` (`[tool.pytest.ini_options]`, `pytest-cov`), `vite.config.js` (Vitest, `jsdom`, `coverage-v8`). Pipeline GitHub Actions (`.github/workflows/cicd.yml`) uruchamia zadania: `backend-lint`, `backend-test` (z bramką `-cov-fail-under=80`), `backend-migrations`, `frontend-lint`, `frontend-test` (z pokryciem) oraz `build-test`. Zielony wynik wszystkich testów jest warunkiem wdrożenia (`deploy`) oraz scalenia PR do `main`, co stanowi ciągłą, weryfikowalną dokumentację wyników testowania.

5 Zarządzanie projektem

5.1 Cele i zakres projektu

Cel. Stworzenie aplikacji webowej PlayLink umożliwiającej szybkie tworzenie i wyszukiwanie sesji do wspólnej gry w niszowe, retro oraz mniej popularne tytuły multiplayer.

Zakres. Projekt obejmuje: interfejs użytkownika, backend obsługujący sesje i komunikację w czasie rzeczywistym oraz funkcje tworzenia, przeglądania, dołączania i opuszczania sesji, czat w obrębie pokoju, bezhasłowe uwierzytelnianie (BIP39), kalendarz/RSVP, panel administratora oraz pełną automatyzację wdrożenia (CI/CD).

Co osiągnięto. Zrealizowano wszystkie zaplanowane wymagania funkcjonalne i nie-funkcjonalne; aplikacja działa na środowisku produkcyjnym. Świadomie pominięto komponent AI/ML (uzgodnione z opiekunem — zob. rozdz. 3.3), przekierowując wysiłek na bezpieczeństwo danych.

5.2 Podział ról i zadań w zespole

Osoba (nr indeksu)	Rola	Wykonywane zadania
Jakub Rusek (247774)	Backend & DevOps	Architektura backendu, autoryzacja, automatyzacja CI/CD (GitHub Actions), strategia wdrożeń, stabilność i dostępność.
Nikodem Nowak (251598)	Backend Developer	Logika serwerowa, implementacja API, komunikacja realtime (WebSocket), zarządzanie sesjami i bazą danych.
Bartosz Kołaciński (251554)	Frontend Developer	Budowa interfejsu użytkownika (UI/UX), integracja frontendu z backendem.
Mateusz Kosowski (251558)	Solution Architect	Projektowanie architektury rozwiązania, spójność decyzji technicznych, koordynacja integracji komponentów.
Jakub Rosiak (251620)	QA & Documentation	Scenariusze testowe, walidacja kluczowych funkcji MVP, utrzymanie dokumentacji projektowej.

Tabela 6: Podział ról i zadań w zespole projektowym.

5.3 Harmonogram realizacji i kamienie milowe

Projekt realizowano w sześciu etapach. Zestawienie etapów wraz z terminami i kamieniami milowymi przedstawia tab. 7.

Tabela 7: Harmonogram realizacji projektu z kamieniami milowymi.

Etap	Termin	Zakres prac	Kamień milowy
1	Tydzień 3	Analiza i założenia: cele i zakres, analiza rozwiązań, wymagania, podział ról, harmonogram, porządek repozytorium.	Zakończenie dokumentacji wstępnej.
2	Tygodnie 4–5	Projektowanie: architektura, struktura frontendu i backendu, główne widoki (user flow), lista zadań w Issues.	Gotowość do implementacji.
3	Tygodnie 6–8	Implementacja MVP: lista i tworzenie sesji, logika dołączania/opuszczania, integracja frontend–backend.	Działające MVP.

Etap	Termin	Zakres prac	Kamień milowy
4	Tygodnie 9–10	Rozszerzenia i realtime: czat, WebSocket, dynamiczna aktualizacja listy pokoi.	Pełna wersja funkcjonalna.
5	Do pocz. czerwca	Testy i poprawki: scenariusze użytkownika, walidacja formularzy, poprawki UI i stabilności, dokumentacja.	Gotowość na termin wstępny.
6	Pocz.–poł. czerwca	Finalizacja: ostatnie poprawki, domknięcie zadań, wersja końcowa, przygotowanie do prezentacji.	Oddanie wersji końcowej.

Podsumowanie kamieni milowych. (1) Tydzień 3 — dokumentacja wstępna; (2) Tydzień 5 — projekt rozwiązania; (3) Tydzień 8 — MVP; (4) Tydzień 10 — pełna wersja funkcjonalna; (5) początek czerwca — gotowość na termin wstępny; (6) połowa czerwca — wersja końcowa.

5.4 Szczegółowy harmonogram: kluczowe wskaźniki efektywności (KPI)

Dla każdego etapu zdefiniowano mierzalne KPI; zestawienie globalne przedstawia tab. 8.

Wskaźnik (KPI)	Cel
Liczba przeanalizowanych rozwiązań konkurencyjnych	≥ 3
Liczba wymagań funkcjonalnych	≥ 10
Liczba wymagań niefunkcjonalnych	≥ 5
Procent ukończonych funkcji MVP (etap 4)	$\geq 90\%$
Liczba działających funkcji realtime	≥ 3
Czas aktualizacji danych (realtime)	$< 1000 \text{ ms}$
Pokrycie testami backendu (bramka CI)	$\geq 80\%$
Liczba otwartych błędów krytycznych przed oddaniem	0
Procent zamkniętych zadań (Issues) do połowy czerwca	$\geq 95\%$
Czas wdrożenia (od push do live)	$\approx 3 \text{ min}$

Tabela 8: Globalne kluczowe wskaźniki efektywności (KPI) projektu.

Wybrane KPI etapowe.

- **Etap 2:** architektura systemu 100%; liczba opisanych głównych widoków ≥ 4 ; liczba zadań implementacyjnych ≥ 10 .
- **Etap 3:** ukończone funkcje podstawowe $\geq 70\%$; zamknięte Issues $\geq 60\%$ zaplanowanych; liczba błędów krytycznych ≤ 5 .

- **Etap 5:** przetestowane główne funkcje $\geq 90\%$; otwarte błędy krytyczne 0; czas reakcji na zgłoszony błąd ≤ 2 dni.
- **Etap 6:** ukończone zadania projektowe $\geq 95\%$; otwarte zadania wysokiego priorytetu ≤ 2 .

5.5 Narzędzia i proces pracy zespołowej

Pracę zorganizowano w oparciu o GitHub: **Issues** (planowanie i podział zadań), **Projects** z tablicą **Kanban** (przepływ *Todo* $\rightarrow$ *Awaiting Review* $\rightarrow$ *Done*) oraz **Milestones** (kamienie milowe). Obowiązywały zasady współpracy (**CONTRIBUTING.md**): rozwój na gałęziach funkcyjnych, obowiązkowy przegląd Pull Requestów oraz konwencja komunikatów commitów. Każdy PR musiał przejść zielony pipeline CI (lint + testy + build) przed scaleniem do `main`, co zapewniało ciągłą jakość i czytelność dokumentacji projektowej.